

DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Design a **hybrid swarm system** combining **swarms-rs** and **ZeroClaw** (already present in Sovereign Machina), mining key logic from **MiroShark**.
- Focus on **pure intelligence boost** and capability enhancement (reasoning depth, knowledge synthesis, pattern extraction, self-improvement, relational memory) in the tri/quad-node Sovereign Machina OS.
- The swarm must be spawned locally from **Node C (Oracle)** as primary entry point, with optional delegation to **Node B (Director)** for high-quality synthesis.
- Maintain: Node A = Mooncake KV buffer (unchanged), zero-trust, physical sovereignty, declarative Nix identity, local-first design, R3D_V01D + VT323 aesthetic.

All requirements are now exhaustive and production-locked.

RESEARCH PHASE – RIGOROUS CROSS-CHECK (2026 frameworks + Sovereign Machina alignment)

- **swarms-rs**: Production-grade Rust-native multi-agent orchestration framework. Strong in low-latency collaboration, memory safety, concurrency, and complex workflows. Excellent for scalable, safe agent teams.
- **ZeroClaw**: Already integrated in Sovereign Machina (Rust-based, minimalist, self-hosted autonomous agent framework). Compact, privacy-focused, and aligned with your Rust sidecars.
- **MiroShark**: Graph-centric swarm engine with archetype personas, Neo4j-style relational memory, background job queuing, live data injection, and counterfactual branching. Strong for intelligence amplification through simulated multi-perspective reasoning.

RECOMMENDATION

Yes — a swarms-rs + ZeroClaw hybrid, mining MiroShark logic, is the optimal path.

This creates a native Rust/Tokio swarm engine that leverages ZeroClaw's existing presence for autonomous execution, swarms-rs for orchestration and safety, and MiroShark patterns

for graph memory and archetype-based intelligence boost. The swarm spawns from Node C (Oracle) for perception input, with results synthesized on Node B (Director).

HYBRID SWARM DESIGN – SOVEREIGN INTELLIGENCE SWARM (Rust/Tokio)

High-Level Architecture

- **Spawn Point:** Node C (Oracle) as primary entry point (voice/screen input + initial orchestration).
- **Execution Layer:** ZeroClaw agents for autonomous, stateful task execution.
- **Orchestration Layer:** swarms-rs for safe, concurrent agent collaboration and workflow management.
- **Memory Layer:** MiroShark-inspired graph memory (MemPalace + SQLite-vec) for relational knowledge and archetype personas.
- **Async Runtime:** Tokio for non-blocking task management and inter-agent communication.

Core Components (in `sovereign-intel-swarm` Rust Crate)

Rust

```
// sovereign-intel-swarm/src/lib.rs
pub struct SovereignSwarm {
    zeroclaw: ZeroClawRuntime,      // Existing ZeroClaw for autonomous
    execution
    swarms: SwarmsRsOrchestrator,   // swarms-rs for safe collaboration
    graph: MemPalaceGraph,          // MiroShark-inspired relational
    memory
    tokio_rt: TokioRuntime,         // Async orchestration
}

impl SovereignSwarm {
    pub async fn new(config: SwarmConfig) -> Result<Self>;

    // Triggered by voice/screen input from Node C
    pub async fn boost_intelligence(&self, input: IntelligenceInput) ->
    Result<SwarmResult>;

    // Background Dream Cycle (self-improvement)
    pub async fn run_dream_cycle(&self) -> Result<()>;
}
```

Mined Logic from MiroShark

- **Archetype Personas:** Create 8–20 lightweight agents with distinct roles (Analyst, Synthesizer, Critic, Historian, Forecaster, etc.).
- **Graph Memory:** Use MemPalace + SQLite-vec for typed nodes/edges and auto-linking (MiroShark-style interaction graph).
- **Background Job Queuing:** Durable Minions queue with live injection and branching for counterfactual reasoning.
- **Intelligence Boost Loop:** Input → Hermes validation → spawn swarm → parallel multi-perspective reasoning → synthesized results fed back to Akashik Artery.

Nix Integration on Node C

```
nix
```

```
services.sovereign-intel-swarm = {  
  enable = true;  
  package = pkgs.sovereign-intel-swarm;  
  after = [ "hermes-supervisor.service" ];  
  wantedBy = [ "multi-user.target" ];  
};
```

Performance on Current Hardware

- Node C (RTX 2060 + 500 GB SSD) can comfortably run 8–20 lightweight agents with Q4 models and Mooncake spillover.
- The 500 GB SSD stores swarm logs and temporary graph state.
- All swarm tasks are provenance-tagged and audited by Ouroboros.

Overall Effect on Sovereign Machina OS

This hybrid swarm turns the tri/quad-node system into a true **self-amplifying intelligence OS**. It boosts pure capability (deeper reasoning, richer synthesis, faster self-improvement) while staying fully local, sovereign, and respectful of the existing roles. Node C becomes the perception + swarm orchestration hub, Node B the final synthesizer, and Node A the shared memory reservoir.

ARCHITECT SIGN-OFF

The swarms-rs + ZeroClaw hybrid, mining MiroShark logic (archetype personas, graph memory, background queuing), is the optimal design for pure intelligence boost. It leverages ZeroClaw's existing presence for autonomous execution, swarms-rs for safe orchestration, and MiroShark patterns for relational depth. Spawning from Node C keeps the stack balanced and respects your current tri/quad-node invariants. This deepens the self-improving nature of Sovereign Machina without touching Node A or Node B core logic. The swarm focuses on capability amplification (reasoning depth, pattern extraction, knowledge synthesis) while remaining lightweight and fully sovereign. When the Hermes Pod is available, it can optionally offload heavier swarm tasks for even better balance.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want the starter Rust/Tokio swarm crate structure (`sovereign-intel-swarm`) with ZeroClaw + `swarms-rs` integration and MiroShark graph memory?
2. Shall I include the exact Nix service definition for spawning the swarm on Node C?

Reply with your priority and I will deliver the concrete starter code. The intelligence swarm is ready to be prototyped.